

Homework Assignment 3

Data Interop: From C++ to the Data-Science Stack

EC327 Summer 2026

Introduction

Assignment goals

By the end of this assignment you will

- have **serialized** an in-memory C++ table to disk in two formats — a human-readable **CSV** text file and a raw **NumPy .npy** binary array — and understood why a real pipeline keeps both around
- understand the **text-vs-binary tradeoff** from having written both by hand: readability and universality on one side, exactness, compactness-per-value, and load speed on the other
- have hit and fixed the **floating-point precision trap** — discovered that a stream’s *default* precision silently corrupts your data, and learned why `max_digits10` is the number that makes a **double** round-trip exactly
- have written a file with an **exact binary layout**: a format header (magic bytes, version, length field, a padded dict), 64-byte alignment, and little-endian IEEE-754 doubles written as raw bytes through `std::ios::binary` and `reinterpret_cast`
- know what **endianness** is, and why `<f8` costs you nothing on the machine in front of you
- have **proven cross-language interoperability**: a Python script using `pandas` and `numpy` loads *both* of your files and confirms they contain the same numbers to the bit
- have met **graceful dependency handling** — a test suite that *skips* (rather than crashes) when a library is missing, and understood why a skip is not a pass

The conceptual goal: a simulation that never leaves C++ memory is a simulation nobody can use. The bridge from “*C++ generated it*” to “*Python analyzes it*” is a **file**, and the file’s format is a **contract**. HW3 is about writing that contract correctly — twice.

Working style

Individual assignment. Talk with classmates about ideas and approaches; write your own code.

AI use is expected and encouraged, per the course’s AI-in-the-learning-loop policy. You must disclose your AI use and you must be able to explain every line of your submission. See the AI disclosure section below.

Due date

Saturday, June 20, 2026, 11:59 PM.

Assignment value

Graded on a 100-point internal scale. HW3 contributes **6.25 course points** (one of four HWs in the 25% HW component).

Late policy

30% flat penalty for any submission up to 1 week late. No credit after that. Documented legitimate excuses (illness, family emergency, etc.) waive the penalty — email the instructor.

Submission

Submit via the course web app at curl.bu.edu/ec327. Submit all required source files plus your captured test output, your reflection, and your AI disclosure, as a single **.zip**.

How this assignment is graded — and the bigger picture

HW1 was the *internal customer with a tight contract*: every signature given, every pitfall named. HW2 was the *external customer with a clear requirements doc*: a solid prose spec and a clean interface, the implementation yours. **HW3 changes what “spec” even means.**

This time the customer hands you a **machine-checkable acceptance test**. The file-format specifications in `csv_writer.h` and `npv_writer.h` are precise enough that a program *on the other side of a language boundary* — `test_interop.py`, written in Python — can load your output and decide, automatically, whether you met the contract. The customer does not care how you laid down the bytes. They care that `numpy.load()` works and the numbers are exact.

That is the framing, from the syllabus: every HW is a **product ship**, and this one is the **vendor relationship** — *the customer cares about outcomes, not the path you took*. The byte-level spec is tighter than HW2’s prose ever was, but the implementation is entirely unscaffolded: the `.cpp` bodies are empty, there are no per-line TODO breadcrumbs, and the one genuinely new skill — writing a **binary** file with an exact layout — you build yourself. Recognizing what kind of spec you have, and working well against a contract you’ll be *tested against by a machine*, is the skill.

Background — what you are building

In HW2 you built a simulation that lives entirely in C++ memory. The moment `main` returns, the data is gone. Real simulations don’t end at `std::cout` — their output feeds a *second stage*: plotting, statistics, machine learning, almost always in Python. Nobody re-implements a Monte Carlo engine in pandas; they load the numbers the C++ side already produced. The join between the two worlds is a **file on disk**.

HW3 is that file. You are given the **HW2 simulation, provided complete** — you do not touch it. It produces a **DataSet**: a table of **K = 5000** finished walkers, each a row of **four double columns**:

column	meaning
x	final x-coordinate
y	final y-coordinate
r_squared	$x^2 + y^2$
distance	$\sqrt{x^2 + y^2}$

Your job: write that table to disk **twice** — once as `data/walks.csv`, once as `data/walks.npy` — such that a Python script using `pandas` and `numpy` loads both and confirms they hold **the same numbers to the last bit**.

Two formats, one table, because the two formats teach opposite lessons:

- **CSV is text.** Universal (every spreadsheet, database, and data-science library reads it), human-readable, diff-able, writable by hand with nothing but `<fstream>`. It is also **lossy by default** — the precision trap you are about to meet — and verbose.
- **.npy is binary.** It stores the raw IEEE-754 bytes: **exact** (no precision to lose), fixed and predictable in layout, and **instant** to load (no parsing). The cost: it is unreadable to a human and *completely* unforgiving about byte layout — one wrong byte and `numpy.load` throws.

Writing both, by hand, is the only way to feel what each one costs.

A note on file size. People will tell you “binary is smaller.” For *this* dataset it isn’t, quite — three of the four columns (x, y, r_squared) are small integers that print short as text, so the CSV (~130 KB) is actually a hair *smaller* than the .npy (~156 KB, since every value is a fixed 8 bytes). Binary’s *guaranteed* wins are **exactness**, **predictable fixed layout**, and **load speed** — not raw size, which depends entirely on your data. On a table of genuine high-entropy floats the binary file would win on size too. Notice this; it’s a real lesson about not repeating folklore you haven’t measured.

Setup

The HW2 simulation is provided complete, so there is no environment work beyond Lab 0. You will receive a starter directory:

```
hw3_data_interop/
├─ CMakeLists.txt      # builds the whole thing – extend it if you add files
├─ DataSet.h           # the table type (WalkRecord, kColumnNames) – PROVIDED, read it
├─ DataSet.cpp         # runs the HW2 sim to fill the table – PROVIDED, don't edit
├─ Walker2D.h          # your HW2 walker – PROVIDED complete this time
├─ Walker2D.cpp        # PROVIDED complete
├─ csv_writer.h        # full spec for Part 1 (read this)
```

— csv_writer.cpp	# empty body — you implement write_csv()
— npy_writer.h	# full .npy format spec for Part 2 (read this carefully)
— npy_writer.cpp	# empty body — you implement write_npy()
— hw3_main.cpp	# the driver — two TODOs to wire up
— smoke_test.py	# build/run plumbing check — run first
— test_interop.py	# the GRADED cross-language test suite
— requirements.txt	# numpy + pandas (the consumer-side libraries)
— ec327_zip.py	# builds + checks your submission zip
— .clang-format	# the course style file

Build and run with:

```
cmake -B build
cmake --build build
./build/hw3_main
```

The starter compiles and runs as-is — it generates the dataset and prints its size, then stops, because the two writers return **false** until you implement them.

The table's column order is **fixed** in `DataSet.h`:

```
inline constexpr const char* kColumnNames[] = {"x", "y", "r_squared", "distance"};
```

Both writers must emit columns in **exactly this order**, or the CSV header and the `.npz` array will disagree with what the Python loader expects — and with each other.

Part 1 — `write_csv`: the text format (25 points)

Goal: write the table as CSV that `pandas.read_csv()` loads cleanly — and that round-trips *exactly*.

`csv_writer.h` declares one function:

```
bool write_csv(const std::string& path, const DataSet& data);
```

The shape is simple:

- open an `std::ofstream` on `path`; if !out, **return false** (a writer that silently fails to open its file is a classic interop bug — the caller checks your return value);
- write the **header line**: the four `kColumnNames`, comma-separated, then `'\n'`;
- for each `WalkRecord`, write `x`, `y`, `r_squared`, `distance` — comma-separated, in that order, **no trailing comma**, a clean newline after the last column;
- return `true`.

A correct run produces a file you can open in any text editor and *read*:

```
x,y,r_squared,distance
-6,8,100,10
-7,17,338,18.384776310850235
-1,17,290,17.029386365926403
6,6,72,8.4852813742385695
-11,-5,146,12.083045973594572
```

...

The trap that makes or breaks this part. A stream's **default precision is 6 significant digits**. Left alone, your distance of 18.384776310850235 is written as 18.3848 — silently rounded, *lossy*, and no longer the number that came out of the simulation. Your CSV will then **disagree with your .npv** (Part 3 checks this to within 10^{-9} and will catch you). The fix is one line before the loop:

```
out << std::setprecision(std::numeric_limits<double>::max_digits10);
```

`max_digits10` is **17** for a double: the smallest number of decimal digits guaranteed to round-trip *any* double exactly. (Its cousin `digits10` is 15 — enough to *display* but **not** to round-trip. Use `max_digits10` here.) You need `<iomanip>` and `<limits>`.

New C++ vocabulary: `std::ofstream` text output · `std::setprecision / std::numeric_limits<double>::max_digits10` · the difference between *displaying* and *round-tripping* a float · returning a status `bool` from an I/O function

Part 2 — `write_npy`: the binary format (35 points)

Goal: write the table as a real NumPy `.npv` v1.0 array — the file `numpy.load()` reads — laying the bytes out exactly right.

This is the new skill of the assignment. `npv_writer.h` declares:

```
bool write_npy(const std::string& path, const DataSet& data);
```

and documents the **complete format** — read it. You will write the table as one 2-D array of shape **(K, 4)**, dtype **float64**, **C-order** (row-major). The layout:

- | | | |
|-----------------|---------------------------------------|-------------------------|
| 1. magic string | "\x93NUMPY" | (6 bytes) |
| 2. version | 0x01 0x00 | (2 bytes: major, minor) |
| 3. HEADER_LEN | little-endian uint16 | (2 bytes) |
| 4. header dict | HEADER_LEN bytes of ASCII | |
| 5. raw data | K*4 little-endian doubles, row by row | |

The header dict is a Python-literal string. For our (K, 4) array:

```
{'descr': '<f8', 'fortran_order': False, 'shape': (5000, 4), }
```

with two rules NumPy enforces: it **ends in a newline '\n'**, and it is **padded with spaces** before that newline so the *total* header — `magic(6) + version(2) + len(2) + dict` — is a **multiple of 64**. (`HEADER_LEN` is the length of that padded, newline-terminated dict.) For our array the whole header lands at **128 bytes**.

The five things you write, in order:

1. the **6 magic bytes**: 0x93 then "NUMPY";
2. the **2 version bytes**: 0x01, 0x00;
3. **HEADER_LEN** as a little-endian uint16 (low byte, then high byte);
4. the **padded header string**;

5. the **data**: each row's four doubles, in column order, as raw bytes — `out.write(reinterpret_cast<const char*>(&value), sizeof(double));`

'<f8' means little-endian, float, 8 bytes. On the lab machines (x86-64 and Apple Silicon) a double is *already* stored little-endian, so writing its raw bytes gives you '<f8' for free — no byte-swapping. (On a big-endian machine you'd have to swap; note this in your reflection.) **Open the stream in binary mode** — `std::ofstream out(path, std::ios::binary)` — so the OS doesn't "helpfully" translate any byte `0x0A` inside your doubles into `\r\n`.

Verify the moment you can write one file:

```
python3 -c "import numpy, sys; a = numpy.load(sys.argv[1]); print(a.shape, a.dtype)" data/walks.npy
```

You want exactly:

```
(5000, 4) float64
```

New C++ vocabulary: `std::ios::binary` · `reinterpret_cast<const char*>` to view an object as raw bytes (a *legitimate* use of `reinterpret_cast`: serialization) · `std::ostream` to build the header · little-endian / IEEE-754 / byte layout · 64-byte alignment / padding · `uint16_t` and `<stdint>`

Part 3 — Wire it together and pass the interop suite (25 points)

Goal: make both writers run, and prove a Python consumer can load what they produced.

First, **wire the two TODOs in `hw3_main.cpp`**: call `write_csv` and `write_npy`, **check each return value**, and report success or failure. A run should look like:

```
Generated 5000 rows x 4 columns (K=5000, N=200).
Wrote data/walks.csv (5000 rows).
Wrote data/walks.npy ((5000, 4) float64).
Now run: python3 test_interop.py
```

Then **install the consumer-side libraries** and run the graded suite:

```
python3 -m pip install -r requirements.txt
python3 test_interop.py 2>&1 | tee test_output.txt
```

`test_interop.py` builds and runs your program, then loads `data/walks.csv` with `pandas` and `data/walks.npy` with `numpy` the way a real downstream user would, and checks seven things:

```
test_csv_file_exists ... ok
test_csv_loads_with_columns ... ok
test_csv_matches_npy ... ok
test_internal_consistency ... ok
test_npy_file_exists ... ok
test_npy_loads_with_shape_and_dtype ... ok
test_statistics_plausible ... ok
```

Ran 7 tests in 0.4s

OK

Two of these are where the assignment's lessons bite:

- **test_csv_matches_npy** loads both files and asserts they agree to within 10^{-9} . If your CSV used the *default* precision (Part 1's trap), the two writers disagree and this **fails**. The cross-check is the machine catching the precision bug for you.
- **A SKIP is not a PASS**. The suite is written to *skip* — not crash — the tests that need a library you didn't install. That's graceful dependency handling, and it's the right design. But **a skipped format earns no credit for that format**. Full credit needs *numpy and pandas* installed and every test **PASSING**, not skipped. Read the suite's output: it tells you, at the top, if something is being skipped.

Capture the passing output into test_output.txt (the *tee* above does this) — it is a required deliverable.

New C++ vocabulary / skills: checking I/O return values at the call site · capturing program output for a deliverable · reading a cross-language test report · the meaning of *skipped* vs *passed* in a test suite

Part 4 — Build hygiene + reflection (15 points)

Goal: keep the project clean, and articulate what the two formats taught you.

Make sure:

- every `.h` has `#pragma once`; every `.cpp` `#includes` its own header;
- `hw3_main.cpp` includes both writer headers;
- `CMakeLists.txt` builds everything (it already lists all five sources — extend it only if you add files);
- the whole thing compiles cleanly under `-Wall -Wextra` and is `clang-format-clean` with the course file.

Write reflection.md — about 100 words (± 20) — addressing:

1. **The precision trap.** What went wrong when you left the CSV at default precision, and **which test caught it**? Why is `max_digits10` the right number?
2. **Text vs binary.** Name one concrete advantage of each that you *experienced* writing them. When would you ship CSV to a consumer, and when `.npz`?
3. **Endianness.** `'<f8'` worked for free on your machine. What would break if a **big-endian** machine read your `.npz`, and what would you have had to change?
4. **The contract.** The customer's acceptance test is `test_interop.py` — you never saw the grader's downstream analysis code. Why is “*the file loads and the numbers are exact*” the **whole** contract?

This is the conceptual deliverable. The point of the homework lives here.

Required submission

Submit a **single .zip file**, uploaded at curl.bu.edu/ec327, containing:

File	What it is
csv_writer.h csv_writer.cpp	Part 1 — the CSV writer
npv_writer.h npv_writer.cpp	Part 2 — the .npv binary writer
hw3_main.cpp	The driver, with both writers wired in
DataSet.h DataSet.cpp	Provided — submit unchanged (the grader's build needs them)
Walker2D.h Walker2D.cpp	Provided — submit unchanged
CMakeLists.txt	Build configuration (may be the unmodified starter)
test_output.txt	Captured output of a passing test_interop.py run
reflection.md	Your ~100-word reflection
ai_disclosure.md	Your AI use disclosure (template below)

Do **not** submit the `build/` directory, the `data/` outputs, or any compiled binaries. Upload **one .zip** (loose files and `.tar.gz` are rejected). Build and check it with the bundled helper — `python3 ec327_zip.py build hw3` — or zip by hand; the site shows the same present/missing checklist against the files inside your zip.

Testing your work — the smoke test

Your starter package includes `smoke_test.py`. Run it before the graded suite:

```
python3 smoke_test.py
```

It checks that the required source files exist, that `cmake -B build && cmake --build build` succeeds, that the binary runs and exits cleanly, and that **both** `data/walks.csv` and `data/walks.npv` were written and are non-empty. It does **not** load or validate the data — that is `test_interop.py`'s job, and that is the graded deliverable.

Passing the smoke test is necessary, not sufficient. It confirms your build plumbing works; it says nothing about whether your CSV round-trips, whether your `.npv` header is laid out correctly, or whether the two writers agree. The instructor will read your code, run `test_interop.py` (with the libraries installed), confirm no test was *skipped*, and read your reflection.

Grading rubric

Part	Pts	Earns full credit	Common deductions
1	25	CSV with the correct header and all 5000 rows in <code>kColumnNames</code> order; full precision so every value round-trips; returns false on open failure	default 6-digit precision (data silently rounded — fails the cross-check) · trailing comma or missing header · wrong column order · ignores the open-failure case
2	35	<code>.numpy</code> with correct magic/version/HEADER_LEN; dict padded to a 64-byte multiple, newline-terminated; HEADER_LEN little-endian; raw little-endian <code>float64</code> data in C-order; <code>numpy.load</code> gives (5000, 4) <code>float64</code> ; stream opened binary	LEN; the 0x0A corruption) · header not padded to 64 or missing the newline · HEADER_LEN wrong endianness/value · wrong shape or dtype · column order disagrees with the CSV
3	25	both writers wired into <code>main</code> with return-value checks; <code>numpy</code> + <code>pandas</code> installed so nothing skips ; all 7 tests PASS ; CSV and <code>.numpy</code> agree to 10^{-9} ; <code>test_output.txt</code> captured	tests SKIP (libraries not installed — no credit for that format) · CSV disagrees with <code>.numpy</code> (precision) · <code>main</code> ignores the writers' return values · <code>test_output.txt</code> missing or shows skips/failures
4	15	clean <code>-Wall -Wextra + clang-format</code> build; <code>reflection.md</code> is 80–120 words and answers all four prompts (precision, text-vs-binary, endianness, the contract); AI disclosure complete	compiler warnings · reflection skips the precision or endianness prompt · no AI disclosure

Style baseline (all parts): compiles under `-Wall -Wextra` with no warnings; `clang-format`-clean with the course file. Note that `clang-tidy` would flag using namespace `std`;, which does not appear in course-distributed code (see L14) — don't write it.

Hints and pitfalls

The precision one-liner (Part 1). Set it once, before the row loop:

```
#include <iomanip>
#include <limits>
// ...
out << std::setprecision(std::numeric_limits<double>::max_digits10);
```

`max_digits10` is 17 for double. With it, `18.384776310850235` is written in full and your CSV matches your `.npz`. Without it, you write `18.3848`, and `test_csv_matches_npz` fails by orders of magnitude more than its 10^{-9} tolerance. The integer-valued columns (`x`, `y`, `r_squared`) survive default precision; `distance` is the one that exposes the bug — which is exactly why a single careless run can *look* fine until the cross-check runs.

Binary mode is not optional (Part 2). `std::ofstream out(path, std::ios::binary);`. The doubles you write contain arbitrary bytes, including `0x0A` (`'\n'`). In text mode on some platforms the runtime rewrites `0x0A` to `0x0D 0x0A`, inserting a stray byte into the middle of a float and corrupting the file in a way `numpy.load` reports as a cryptic shape error. macOS and Linux happen not to translate, but write `std::ios::binary` anyway — it's correct everywhere and the grader may run anywhere.

The 64-byte padding arithmetic. The total header (`10 + dict.size() + 1` — the `+1` is the trailing newline) must be a multiple of 64. Compute the padding and apply it before the newline:

```
size_t unpadded = 10 + dict.size() + 1;           // 10 = magic(6)+version(2)+len(2)
size_t pad = (64 - (unpadded % 64)) % 64;         // outer %64 handles already-aligned
dict.append(pad, ' ');
dict.push_back('\n');
uint16_t header_len = static_cast<uint16_t>(dict.size());
```

Writing HEADER_LEN little-endian. Low byte first, then high byte:

```
out.put(static_cast<char>(header_len & 0xFF));
out.put(static_cast<char>((header_len >> 8) & 0xFF));
```

C-order means row-major. Write row 0's four doubles, then row 1's, and so on — the natural loop. That is what `fortran_order: False` promises the reader. Match the column order to `kColumnNames`: `x`, `y`, `r_squared`, `distance`.

`reinterpret_cast` is correct here. Serializing an object to its raw bytes is one of the handful of legitimate uses of `reinterpret_cast`. You are not lying about a type; you are asking for the object's byte representation, which is exactly what a binary file stores:

```
out.write(reinterpret_cast<const char*>(&value), sizeof(double));
```

Don't byte-swap on the lab machines. x86-64 and Apple Silicon are little-endian, so the native byte order *is* '<f8'. Just write the bytes. The reflection asks where you'd swap on a big-endian host — that's the conceptual point, not something you implement here.

A skipped test is the suite saying “I couldn't check this.” If you see skipped in the output, you did not install numpy/pandas. Run `python3 -m pip install -r requirements.txt` and re-run. The rubric scores a skipped format as zero — install the libraries.

AI disclosure (required)

Submit `ai_disclosure.md` with the following template filled out. ~100 words total.

`## AI use disclosure for HW3`

`### Roles used`

(check all that apply)

```
[ ] Pair programmer – AI drafted or completed code with me
[ ] Coach – AI explained concepts to me
[ ] IT aide – AI helped me with environment / compiler / build issues
[ ] Code reviewer – AI critiqued code I had written
[ ] Test-case generator – AI suggested edge cases
[ ] Spec clarifier – AI helped me unpack ambiguities in the assignment
[ ] Rubber duck – I explained the problem to AI and figured it out mid-explanation
[ ] Cross-language translator – AI showed me C++ equivalents of Python idioms
```

`### What I accepted, what I rejected`

(2–3 sentences. Where did AI help? Where did it suggest something wrong or unhelpful, and how did you catch it?)

`### Confidence`

(1 sentence. Can you walk through every line of your submission and explain what it does and why?)

The disclosure is the practice of the AI policy, not a hurdle. There is no penalty for *using* AI heavily; there is a penalty for using it without disclosing, or for submitting code you cannot explain. A `.npz` writer is exactly the kind of thing AI produces instantly and correctly — which is *why* the points live in your reflection (explaining the precision bug and the endianness question in your own words) and the weekly oral check. See the course AI policy for the full statement.

What comes next

You can now get data *out* of a C++ program and into the tools a data scientist actually uses — the single most practical skill in this batch. **HW4** turns inward again: you will

implement sorting and searching, make them generic with **templates**, and **benchmark** them inside your own simulation, watching the $O(n \log n)$ vs $O(n^2)$ gap open up on real timings. Keep your writers — being able to dump a dataset to CSV or **.npy** is how you'll capture those benchmark numbers for plotting.